



Alliance School of Advanced Computing

Course Name: Graph Analytics

Course Code: E1ITC 302

Assignment No: 3

Name of the Student: B.Charitha sree

Registration Number: 2411021060285

A. Title of The Problem:

Minimum Weight Path Finder in a Grid

B. Problem Statement in details:

The project focuses on finding the **minimum-cost path between two cells in a weighted grid**. Each cell contains a specific weight, and movement is allowed in four directions: up, down, left, and right. For multiple queries, the system must determine the path with the minimum total weight between the given source and destination cells. Since the grid may contain a large number of columns and queries, an efficient approach using **Dijkstra's Algorithm with Divide and Conquer** is required to reduce computation time and provide optimal paths efficiently.

C. HackerRank/Leetcode Link of the problem (If applicable):

<https://www.hackerrank.com/challenges/dijkstrashortreach/problem>

D. Student's Github link of report and code upload, along with the readme file:

<https://github.com/Shivani160207/Graph-Analysis/tree/main/assignment%202>

E. Solution Steps/Algorithm:

1. Read the grid, its cell weights, and the queries.
2. Divide the grid using the **middle column**.
3. Apply **Dijkstra's Algorithm** from each cell of the middle column.
4. Calculate the minimum path cost for queries passing through the middle column.
5. Subtract the middle cell weight to avoid double counting.
6. Recursively process queries in the left and right sections.
7. Store the minimum cost for each query.
8. Print the minimum-weight path cost for every query.

F. Code developed:

```
import sys
```

import

heapq

```
input =
sys.stdin.readline

n, m = map(int, input().split())

grid = []
for _ in range(n):
    grid.append(list(map(int, input().split())))

q = int(input())

queries = []
for _ in range(q):
    r1, c1, r2, c2 = map(int, input().split())
    queries.append((r1, c1, r2, c2))

INF = 10**18
answer = [INF] * q

def dijkstra(L, R, start_row, mid):
    width = R - L + 1

    dist = [[INF] * width for _ in range(n)]

    start_col = mid - L

    dist[start_row][start_col] = grid[start_row][mid]

    pq = []
    heapq.heappush(
        pq,
        (grid[start_row][mid], start_row, mid)
    )

    while pq:
        current_dist, r, c = heapq.heappop(pq)

        if current_dist != dist[r][c - L]:
            continue
```

Up
nr =

```
r - 1
nc = c

if nr >= 0:
    new_dist = current_dist + grid[nr][nc]

    if new_dist < dist[nr][nc - L]:
        dist[nr][nc - L] = new_dist
        heapq.heappush(pq, (new_dist, nr, nc))

# Down
nr = r + 1
nc = c

if nr < n:
    new_dist = current_dist + grid[nr][nc]

    if new_dist < dist[nr][nc - L]:
        dist[nr][nc - L] = new_dist
        heapq.heappush(pq, (new_dist, nr, nc))

# Left
nr = r
nc = c - 1

if nc >= L:
    new_dist = current_dist + grid[nr][nc]

    if new_dist < dist[nr][nc - L]:
        dist[nr][nc - L] = new_dist
        heapq.heappush(pq, (new_dist, nr, nc))

# Right
nr = r
nc = c + 1

if nc <= R:
    new_dist = current_dist + grid[nr][nc]

    if new_dist < dist[nr][nc - L]:
```

```
dist[nr][nc - L] = new_dist
    heapq.heappush(pq, (new_dist, nr, nc))
```

```
return dist
```

```
def solve(L, R, query_list):
```

```
    if not query_list:
        return
```

```
    mid = (L + R) // 2
```

```
    all_distances = []
```

```
    for row in range(n):
        d = dijkstra(L, R, row, mid)
        all_distances.append(d)
```

```
    for qi in query_list:
```

```
        r1, c1, r2, c2 = queries[qi]
```

```
        for row in range(n):
```

```
            d1 = all_distances[row][r1][c1 - L]
            d2 = all_distances[row][r2][c2 - L]
```

```
            value = d1 + d2 - grid[row][mid]
```

```
            if value < answer[qi]:
                answer[qi] = value
```

```
    if L == R:
        return
```

```
    left_queries = []
    right_queries = []
```

```
    for qi in query_list:
```

c1, r2,
queries[qi]

r1,
c2 =

```
    if c1 < mid and c2 < mid:
        left_queries.append(qi)

    elif c1 > mid and c2 > mid:
        right_queries.append(qi)

    if L <= mid - 1:
        solve(L, mid - 1, left_queries)

    if mid + 1 <= R:
        solve(mid + 1, R, right_queries)

solve(0, m - 1, list(range(q)))

for value in answer:
    print(value)
```

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

✔ Test case 0

✔ Test case 1

✔ Test case 2

✔ Test case 3

✔ Test case 4

✔ Test case 5

✔ Test case 6

Compiler Message

Success

Input (stdin)

```
1 3 5
2 0 0 0 0
3 1 9 9 1
4 0 0 0 0
5 3
6 0 0 2 4
7 0 3 2 3
8 1 1 1 3
```

Download

H. Observation:

The algorithm successfully finds the **minimum-weight path** between the given source and destination cells. Using **Dijkstra's Algorithm with Divide and Conquer** reduces unnecessary computations and efficiently handles multiple queries in the weighted grid.